Sprint 6 · Reto «Datos sucios, cliente impaciente»

Ficha completa del reto, para consultar. Para trabajar con ella abierta, usad el reto en una página.

- Volver al README del sprint · README del sprint en PDF · ecofriendly
- **Esta ficha en PDF:** a color · ecofriendly, para imprimir
- Si es vuestro primer reto: Cómo se trabaja un reto.

RETO Ficha del reto

1 Qué os pide el cliente

Qué os pide el cliente. TurisData Canarias os manda por correo un fichero de reservas, `reservas_sucio.csv`, **exportado a mano desde varios sistemas**. El gerente quiere un primer modelo que prediga qué reservas se cancelan «para ayer», pero **no se fía de los datos** (con razón). Os pide dos cosas: un **informe corto** que diga **qué está mal en los datos y por qué**, y un **proceso automático** (una función y un *pipeline*, es decir, una cadena de pasos que se ejecuta siempre igual) que deje los datos listos cada vez que le llegue un fichero así, con un primer modelo sencillo de referencia (un *baseline*) para saber si merece la pena seguir.

Cómo se trabaja. Los datos reales siempre vienen sucios (fechas escritas de varias maneras, países con distintos nombres, números imposibles, filas repetidas, huecos…) y limpiarlos es entre el 60 y el 80 % del trabajo en un proyecto de IA. No hace falta experiencia previa: ya habéis practicado cada técnica en los cuadernos del sprint, y aquí las juntáis en un caso real. Regla de oro: **toda corrección va escrita en código**, nunca editando el CSV a mano, para que se pueda repetir. En clase tenéis 10 minutos de planning (día 1), **80 minutos de reto** (30 el día 1 y 50 el día 2) y **35 de entrega, review y retrospectiva** (día 2); lo que no quepa va a casa o a la 6.ª hora. El equipo decide hasta dónde llega: mínimo, estándar o extra.

2 · Recursos del reto

Recurso	Para qué lo usaréis	Dónde está
Google Colab (Python con `pandas`, `matplotlib`, `seaborn` y `scikit-learn`, ya instalados)	Escribir y ejecutar `reto_S06.ipynb`	Colab, con vuestra cuenta de Google. Solo librerías del curso
El CSV sucio `reservas_sucio.csv`	Los datos del cliente (sintéticos, sin datos personales). **No se edita a mano**	Los cuadernos del sprint lo descargan solos con la función `dato(...)` de su primera celda; cópiala en vuestro cuaderno
La plantilla del informe `informe_calidad_plantilla.md`	Tabla de defectos, antes y después, preguntas al cliente, IA responsable y revisión con la rúbrica	En GitHub: abrid el archivo y copiad el texto con el botón *Raw*
Cuadernos del sprint: `S06_01` (EDA y `resumen_calidad`) y `S06_02` (limpieza y mini-reto `limpiar_reservas`)	Las técnicas y el código de partida para el inventario y la limpieza	Colab; su código explicado
Cuadernos del sprint: `S06_03` (*pipeline*) y `S06_04` (*baseline*, métricas y validación cruzada)	El código de partida para el *pipeline* y la comparación con el modelo «tonto»	Colab; su código explicado: `S06_03` y `S06_04`

Recurso	Para qué lo usaréis	Dónde está
Cómo entregar en GitHub	Subir la carpeta `sprint-06/` (incluido el cuaderno de Colab, apartado 6 de la guía)	Guía de entrega
Autoevaluación y coevaluación	Cada persona rellena su hoja	Hoja del sprint
Un asistente de IA (opcional)	Entender errores y pedir pistas	Con las reglas de la «Ayuda 1» del paso a paso

3 · Qué entregar

Este apartado responde a **dos preguntas distintas**, una tabla para cada una:

Tabla A · Los ficheros: qué ficheros hay, de dónde sale cada uno (ya hecho, lo modificáis, lo creáis o lo genera el programa) y **cuáles se suben y adónde**.

Tabla B · Los requisitos: qué tiene que cumplir lo que entregáis y hasta dónde llegar (mínimo, estándar, extra). **Es lo que se evalúa.**

En total, al final del sprint se suben a `sprint-06/` : 3 ficheros del equipo (`informe_calidad.md` , `reto_S06.ipynb` , `README.md`) **+ 1 autoevaluación y coevaluación por persona**, es decir, **3 + tantos ficheros como personas haya en el equipo** (un equipo de 3 sube 6). «El reto» no es un fichero aparte: son esos tres ficheros. Además, el CSV va a `data/` si aún no está. La *review* y la *retro* se hacen de palabra: no se suben.

Tabla A · Los ficheros: de dónde sale cada uno y cuáles se suben

Todo lo que se sube va al repositorio privado del equipo (`ifcd107-equipo-…`), en la carpeta **`sprint-06/`**, salvo el CSV.

Fichero	Origen	Qué hacéis vosotros	¿Se sube?
`informe_calidad.md`	**Lo creáis a partir de una plantilla.** Os damos `informe_calidad_plantilla.md` con los apartados y la tabla de defectos	Copiar el texto de la plantilla en un archivo nuevo llamado `informe_calidad.md` y rellenarlo, incluida la sección «Revisión con la rúbrica»	**Sí** · `sprint-06/`
`reto_S06.ipynb`	**Lo creáis.** No hay plantilla: abrís un cuaderno nuevo en Colab y reutilizáis el código de `S06_01` a `S06_04`	Inventario de defectos, función `limpiar_reservas` , partición *train/test*, *pipeline*, *baseline* y validación cruzada, en este orden	**Sí** · `sprint-06/`
`README.md`	**Lo modificáis.** El repositorio del equipo ya trae un `README.md` en `sprint-06/` (si no está, lo creáis)	Rellenarlo: cómo ejecutar el cuaderno, versiones de las librerías, quién hizo qué y uso de la IA	**Sí** · `sprint-06/`
`informe_calidad_plantilla.md`	**Ya hecho.** Es solo el punto de partida del informe	Nada: se copia su contenido en `informe_calidad.md`	**No**: se sube ya rellena, con el nombre `informe_calidad.md`

Fichero	Origen	Qué hacéis vosotros	¿Se sube?
`reservas_sucio.csv`	**Ya hecho.** Los datos del cliente (sintéticos)	Nada: **no se edita a mano**; toda corrección va en el código	**Sí**, sin cambios · `data/` (si aún no está)
Autoevaluación y coevaluación (`autoevaluacion_nombre.md`)	**Ya hecha, en blanco**	Cada persona copia la plantilla con su nombre y la rellena; la coevaluación va **dentro del mismo fichero**	**Sí**, **una por persona** (no una por equipo) · `sprint-06/`

Tabla B · Los requisitos: qué tiene que cumplir y hasta dónde llegar

Requisito	Qué se pide exactamente	Nivel	Dónde se comprueba
Tabla de defectos	Al menos **6 defectos**, cada uno con qué es, dónde está, cuántas filas afecta, causa probable, tratamiento y por qué. Los **tres más graves, destacados**	Mínimo	`informe_calidad.md` · apartado 2
Función de limpieza	**Una sola** función `limpiar_reservas(df)` con docstring que devuelve la tabla limpia **y un resumen de lo que ha cambiado**. No toca el CSV original y no descarta ninguna fila sin contarla y justificarla	Mínimo	`reto_S06.ipynb` y `informe_calidad.md` · apartado 3
Cuaderno que se ejecuta entero	*Entorno de ejecución → Reiniciar y ejecutar todo* termina sin errores	Mínimo	`reto_S06.ipynb`
README	Cómo ejecutar el cuaderno y en qué orden, y quién hizo qué. En el mínimo, versión corta	Mínimo	`README.md`
Entrega en el repositorio	Los tres ficheros en `sprint-06/`, subidos desde el navegador por personas distintas (cada subida es un *commit* con su nombre)	Mínimo	Carpeta `sprint-06/` y su historial
Pipeline sin fuga de datos	Partición *train/test* y `Pipeline` con `ColumnTransformer` que **imputa, escala y codifica ajustando solo con *train***. La función de limpieza solo aplica reglas fijas (no calcula medias ni medianas)	Estándar	`reto_S06.ipynb`
Baseline y modelo comparados	`DummyClassifier` y un modelo (por ejemplo, regresión logística) con validación cruzada, y una frase con **media ± desviación** de la métrica elegida	Estándar	`reto_S06.ipynb` y `informe_calidad.md` · apartado 4
Preguntas al cliente	Al menos 3 preguntas sobre lo que los datos no permiten decidir (por ejemplo, qué significa un precio de 0 €)	Estándar	`informe_calidad.md` · apartado 5
IA responsable	Respuesta concreta a «¿qué pasa si el país de origen se imputa mal? ¿podría el modelo discriminar por nacionalidad? ¿hay columnas que no deberían usarse?» y, en el README, qué se pidió a la IA y qué se revisó	Estándar	`informe_calidad.md` · apartado 6 y `README.md`
README con versiones	Versión de Python y de `pandas` y `scikit-learn`, y el orden de ejecución	Estándar	`README.md`
Revisión y autoevaluación	Alguien que **no escribió esa parte** revisa informe, cuaderno y README con la rúbrica (justo debajo de estas tablas) y anota nivel y prueba en el informe; cada persona sube su hoja de autoevaluación y coevaluación	Estándar	`informe_calidad.md` (sección «Revisión con la rúbrica») y `sprint-06/`

Requisito	Qué se pide exactamente	Nivel	Dónde se comprueba
Extras	Tabla con 8–10 defectos · partición temporal en lugar de aleatoria, justificada · explicar (o probar) qué pasaría si se ajustara el escalado con todos los datos	Extra	`informe_calidad.md` y `reto_S06.ipynb`

Rúbrica del reto · cómo se puntúa y cómo se revisa

La usan **dos veces**: el equipo, para revisar su propio trabajo antes de entregar (es lo que dice la *Definition of Done*), y el docente, para puntuar. Se mira **cada criterio por separado** y se pone el nivel de la columna cuyo texto **se cumple entero**: si lo de un nivel se cumple a medias, es el nivel de abajo. **0** significa que no se ha entregado nada de ese criterio.

Orientación: lo que pide el nivel mínimo sale en un **2**, el estándar en un **3** y los extras en un **4**.

Criterio	1 · Empezado	2 · Mínimo	3 · Estándar	4 · Extra
Corrección técnica (¿funciona?)	Hay código, pero el cuaderno falla al ejecutarlo entero o se ha corregido el CSV a mano	`limpiar_reservas` es una sola función con docstring, devuelve tabla limpia y resumen, y trata en código todos los defectos de la tabla	Además, el `Pipeline` imputa, escala y codifica **ajustando solo con *train***, y el baseline y el modelo se comparan con validación cruzada	Además, partición temporal justificada o una prueba de qué cambia si se ajusta con todos los datos
Reproducibilidad (¿lo puede repetir otra persona?)	Solo funciona si se ejecutan las celdas sueltas en cierto orden	*Reiniciar y ejecutar todo* funciona y el README dice cómo ejecutarlo, pero faltan versiones o el orden	Otra persona obtiene **las mismas cifras** siguiendo solo el README, con versiones de las librerías y semilla fijada	Además, el README indica Python, librerías, ficheros necesarios y de dónde sale el CSV, y se ha probado en un cuaderno nuevo de Colab
Análisis y comunicación al cliente (¿lo entiende quien no programa?)	El informe dice «hay nulos» sin más: no dice dónde, cuántas filas ni por qué	Tabla de al menos 6 defectos con dónde, cuántas filas, causa probable y tratamiento, y los tres más graves destacados	Además, cada tratamiento está justificado con su riesgo, hay resumen para el gerente sin jerga, 3 preguntas al cliente y una frase del baseline con media ± desviación	Además, 8–10 defectos y una recomendación para el cliente con los límites del modelo explicados
IA responsable (¿se sabe qué hizo la IA y qué comprobasteis?)	No se menciona la IA ni el riesgo de usar el país de origen	Dice que se ha usado IA, o menciona un riesgo, pero en genérico	Responde con ejemplos del dataset a la pregunta del país de origen y de las columnas que no deben usarse; dice **qué se pidió a la IA, qué se revisó y qué se cambió**	Además, aplica una medida y la evalúa (por ejemplo, el modelo sin `pais_origen` frente al modelo con él) y cuenta un fallo concreto de la IA que detectasteis

Cómo se revisa (unos 10 minutos, antes de subir; lo hace **una persona que no escribió esa parte**):

1. **Ejecutar** `reto_S06.ipynb` con *Entorno de ejecución → Reiniciar y ejecutar todo*, siguiendo solo lo que dice el README. Anotar si algo falla y qué cifras salen.
2. **Leer el informe como si fuera el gerente**: ¿se entiende qué está mal en los datos, cuántas filas afecta y qué se ha hecho con cada defecto? Subrayar lo que no se entienda.
3. **Mirar el código**: ¿`limpiar_reservas` es una sola función con docstring? ¿Algún `fit` o `fit_transform` usa datos de *test* o la tabla completa? ¿Hay algo que no sepáis explicar? Que quien lo escribió lo explique.
4. **Poner un nivel (1 a 4) en cada criterio, con una frase de prueba** (por ejemplo: «3 · el pipeline se ajusta solo con train y hay media ± desviación, pero falta la versión de scikit-learn en el README»). Sin prueba escrita, el nivel no vale.

5. **Apuntar el resultado** en la sección «Revisión con la rúbrica» del `informe_calidad.md` y corregir lo más fácil de arreglar. Lo demás se anota como mejora.

Cada persona, además, se puntúa a sí misma en su hoja de autoevaluación y coevaluación con **esta misma rúbrica**.

---

4 · Guía paso a paso

Son cinco pasos, de **90 minutos en total** (10 de planning y 80 de reto). Cada uno dice **qué hacéis** (la lista corta), **cómo se hace** con el detalle, **qué deberíais ver** y **cuándo está hecho**. Las ayudas (IA, errores, atascos) están al final. Si el tiempo aprieta, completad el nivel mínimo y dejad lo demás para casa.

Palabras que van a aparecer en este reto

Palabra	Qué significa, en sencillo
Defecto de calidad	Un problema del dato: huecos (nulos), duplicados, valores imposibles, formatos mezclados…
Imputar	Rellenar un hueco con un valor razonable (la mediana, la categoría más frecuente…)
Outlier (valor atípico)	Un valor muy raro. A veces es un error y a veces es real: hay que decidir
Train / test	Los datos con los que entrena el modelo (*train*) y los que se guardan para evaluarlo (*test*)
Pipeline	Una cadena de pasos de preparación y modelo que se ejecuta siempre igual
Fuga de datos	Cuando información del *test* «se cuela» en el entrenamiento y los resultados parecen mejores de lo que son
Baseline y `DummyClassifier`	El modelo de referencia, y un «modelo tonto» (por ejemplo, responde siempre la clase más frecuente) que es el suelo con el que comparar
Validación cruzada	Entrenar y evaluar varias veces con particiones distintas para tener una medida más fiable (media ± desviación)

Paso 1 · Planning y reparto de roles (día 1, 10 minutos)

1. Elegir **Scrum Master y responsable técnico** de este sprint (no pueden repetir quienes lo fueron en el anterior).
2. Leer los apartados 1 y 3 de esta ficha y decidir hasta qué nivel queréis llegar.
3. Pasar al tablero las tareas y decidir quién empieza cada una.

Cómo se hace

- **Roles.** El Scrum Master cuida el tiempo y el tablero; el responsable técnico vigila que el cuaderno se pueda repetir. Todo el equipo trabaja en el contenido.
- **Tablero.** Una tarjeta por tarea: inventario de defectos, función de limpieza, *pipeline*, *baseline*, informe, README.
- **Quién empieza qué.** Todo el equipo debe entender todo, así que quien no programa una parte la revisa. Un reparto posible para los pasos 2 a 4:

Persona	Empieza por	Revisa
A	Inventario de defectos y tabla del informe	La función de limpieza
B	Función `limpiar_reservas` y partición *train*/*test*	El *pipeline*
C	*Pipeline*, *baseline* y validación cruzada	El inventario de defectos
D (si hay 4)	README y preguntas al cliente	El informe entero

Qué deberíais ver. Seis tarjetas en el tablero, con una persona asignada cada una, y el nivel al que apuntáis anotado.

Hecho cuando: los roles están elegidos y cada persona sabe con qué tarea empieza.

Paso 2 · Inventario de defectos (día 1, 30 minutos)

1. Abrir un cuaderno nuevo en Colab (`reto_S06`), cargar `reservas_sucio.csv` y pasar `resumen_calidad`.
2. Buscar defectos con tablas y gráficos, categoría por categoría.
3. Rellenar **una fila de la tabla del informe por cada defecto** (mínimo 6).
4. Apuntar las preguntas que haríais al cliente.

Cómo se hace

- **Cargar el CSV.** Copiad en la primera celda la función `dato(...)` de `S06_02` (descarga el fichero si no está) y usad `crudo = pd.read_csv(dato("reservas_sucio.csv"))`. Guardad siempre esa tabla original **sin tocarla**: se limpia una copia.
- **Mirar el estado de cada columna.** Copiad `resumen_calidad` de `S06_01` o `S06_02` (tipo, nulos, porcentaje de nulos y valores distintos por columna) y mirad también `crudo.shape` y `crudo.describe()`.
- **Buscar por categorías.** Estas son las ocho familias de defecto que debéis revisar (están también en la plantilla del informe):

Categoría	Cómo se busca	Pista de pandas
Duplicados	Contar filas repetidas	`crudo.duplicated().sum()`
Tipos incorrectos	Ver si una columna numérica o de fecha aparece como texto	`crudo.info()` o la columna `tipo` de `resumen_calidad`
Formatos de fecha	Mirar cómo se escriben las columnas de fecha	Ver varias filas y probar `pd.to_datetime(..., format=..., errors="coerce")`
Categorías inconsistentes	Listar los valores distintos de las columnas de texto	`crudo[columna].value_counts()`
Valores imposibles	Mirar mínimos y máximos de las columnas numéricas	`crudo.describe()`, histogramas y diagramas de caja
Outliers	Distinguir un valor extremo pero posible de uno imposible	Diagrama de caja; ante la duda, es una pregunta para el cliente
Nulos	Ver dónde faltan datos y cuántos: **mirad todas las columnas**, también las de texto como `regimen`	`crudo.isna().sum()`
Incoherencias entre columnas	Cruzar columnas que deberían cuadrar	Recalcular una columna a partir de otras y comparar

- **Rellenar la tabla.** Por cada defecto, una fila de `informe_calidad_plantilla.md`. Un ejemplo (inventado) de cómo debe quedar:

Campo	Ejemplo
Qué	Fechas de llegada en dos formatos (`2025-07-03` y `03/07/2025`)
Dónde	Columna `fecha_llegada`
Filas	183 de 2.000
Causa probable	Dos sistemas de origen exportan con formato distinto
Tratamiento	Convertir todas a formato ISO
Por qué (y riesgo)	No se pierde información; el riesgo es confundir día y mes

- **Preguntas al cliente.** Lo que no podéis decidir solos se apunta como pregunta (por ejemplo: «¿un precio de 0 € es cortesía o un error?»), con lo que haréis mientras tanto. Pista de cuaderno: `S06_01` y `S06_02`.

Qué deberíais ver. Un fichero de **1545 filas y 18 columnas**, una tabla de `resumen_calidad` con nulos en **cuatro columnas** (`pais_origen`, `regimen`, `adultos` y `precio_noche`) y alguna columna con un tipo que no es el esperado, y en el informe una tabla con **al menos 6 filas**. La plantilla `informe_calidad_plantilla.md` da una columna para cada cosa: número, defecto, columna(s), filas afectadas, cómo lo detectasteis, causa probable, tratamiento aplicado y por qué (y riesgo). Marcad los tres defectos más graves.

Hecho cuando: la tabla tiene 6 o más defectos con sus datos completos y hay al menos una pregunta para el cliente. Si no encontráis más, mirad la «Ayuda 3».

Paso 3 · Función de limpieza y partición train/test (día 2, 15 minutos)

1. Escribir `limpiar_reservas(df)`: una función con docstring que aplica **todos** vuestros tratamientos.
2. Comprobar el antes y el después.
3. Separar *train* y *test*.

Cómo se hace

- **La función.** Recibe la tabla sucia y devuelve **dos cosas**: la tabla limpia y un resumen de lo que ha cambiado (por ejemplo, un diccionario con filas antes y después, duplicados quitados y nulos nuevos). Empezad por la función del mini-reto de `S06_02` y adaptadla a los defectos de vuestra tabla. Trabaja sobre una **copia** (`crudo.copy()`): el original no se modifica.
- **Reglas fijas, no estadísticas.** La función solo aplica reglas que no dependen de los datos (convertir tipos, unificar categorías, marcar como nulo lo imposible). **No calcula medias, medianas ni categorías frecuentes**: eso se aprende en el *pipeline* del paso 4, solo con *train*.
- **Ninguna fila se pierde sin avisar.** Cada fila que eliminéis (por ejemplo, un duplicado) se cuenta y se justifica en el informe. Un valor imposible se convierte en nulo en lugar de borrar la fila entera.
- **Antes y después.** Haced una tabla pequeña con filas, duplicados, nulos por columna y valores distintos de las categorías antes y después. Si una corrección no se nota, no sirve. Es el apartado 3 del informe.
- **Partición.** Con la tabla limpia, `train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)`, donde `y` es la columna `cancelada`. Decidid qué columnas entran en `X`: pregunta para cada una, «¿se conoce **al reservar**?». Un identificador como `id_reserva` no sirve para predecir. Pista de cuaderno: `S06_02` (mini-reto) y `S06_03` (partición).

Qué deberíais ver. La tabla limpia sin duplicados exactos (con el CSV del curso, **1545 filas antes y 1500 después**), con las fechas como fechas, los valores de `canal` y `pais_origen` unificados y los valores imposibles convertidos en nulos; el resumen de cambios; y dos tablas, *train* y *test*, con la misma proporción de reservas canceladas.

Hecho cuando: `limpiar_reservas(crudo)` devuelve la tabla limpia y el resumen sin cambiar `crudo`, y *train* y *test* están separados antes de calcular nada que se aprenda de los datos.

Paso 4 · Pipeline y baseline (día 2, 15 minutos)

1. Construir un `Pipeline` que impute, escale y codifique.
2. Añadir un modelo de referencia y un `DummyClassifier`.
3. Compararlos con validación cruzada y escribir la frase del resultado.

Cómo se hace

- **Columnas.** Separad las numéricas de las de texto (categóricas) de `X`. Ojo: `regimen` es de texto y tiene huecos; irá por la rama categórica, que los rellena con el valor más frecuente.
- **El *pipeline*.** Un `ColumnTransformer` con dos ramas: numéricas con `SimpleImputer(strategy="median")` y `StandardScaler()`; categóricas con `SimpleImputer(strategy="most_frequent")` y `OneHotEncoder(handle_unknown="ignore")`. Después, un `Pipeline` que une el preprocesado con `LogisticRegression(max_iter=1000)`. Como todo va dentro del `Pipeline`, se ajusta **solo con los datos que le pasáis a `fit`**: eso evita la fuga.
- **El modelo «tonto».** El mismo `Pipeline`, pero con `DummyClassifier(strategy="prior")` en el último paso.

- **Validación cruzada.** `cross_val_score(modelo, X_train, y_train, cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=42), scoring=...)`. Elegid una métrica que mire las cancelaciones (`"f1"` o `"roc_auc"`) y decid cuál y por qué. **Con `"roc_auc"`, el baseline debe salir en torno a 0,5**.
- **La frase.** «Nuestro modelo obtiene un F1 medio de … ± …, frente a … del modelo tonto. Eso significa que …». Va en el apartado 4 del informe.
- Pista de cuaderno: `S06_03` (*pipeline*) y `S06_04` (*baseline* y validación cruzada).

Qué deberíais ver. Una tabla de dos filas (modelo tonto y vuestro modelo) con la media y la desviación de la métrica en los 5 *folds*, y vuestro modelo por encima del tonto. Si no lo supera, algo falla: mirad la «Ayuda 2».

Hecho cuando: el cuaderno tiene el *pipeline*, la tabla de resultados y la frase con media ± desviación, y sabéis explicar por qué el escalado y la imputación no usan el *test*.

Paso 5 · Informe, README y revisión con la rúbrica (día 2, 20 minutos)

1. Completar `informe_calidad.md` a partir de la plantilla.
2. Rellenar el `README.md` y comprobar *Reiniciar y ejecutar todo*.
3. Pedir a alguien que no escribió esa parte que revise con la rúbrica y anote el resultado.

Cómo se hace

- **El informe** (1–2 páginas). Copiad la plantilla en `informe_calidad.md` y completad: resumen para el gerente (máximo 5 líneas), tabla de defectos, antes y después, *pipeline* y modelo de referencia, al menos 3 preguntas al cliente, IA responsable y cómo reproducirlo. **Para el nivel mínimo** bastan el resumen, la tabla de defectos (6 o más) y el antes y después; para el estándar, completad también el resto.
- **IA responsable.** Respondedlo con ejemplos del dataset: ¿qué pasa si el país de origen se imputa mal? ¿Podría el modelo discriminar por nacionalidad? ¿Hay columnas que no deberían usarse?
- **El README.** Cuatro cosas, en pocas líneas: qué hace el cuaderno; cómo ejecutarlo y en qué orden (*abrir en Colab y ejecutar de arriba abajo*); las versiones, que sacáis con `import sys, pandas, sklearn` y `print(sys.version, pandas.__version__, sklearn.__version__)`; y quién hizo qué y **qué le pedisteis a la IA y qué revisasteis**.
- **Reiniciar y ejecutar todo.** En Colab, *Entorno de ejecución → Reiniciar y ejecutar todo*. Si falla, hay celdas que dependen de un orden: arreglad antes de seguir.
- **La revisión con la rúbrica (5 minutos).** Quien revisa sigue los cinco pasos de «Cómo se revisa» (apartado 3) y rellena la sección «Revisión con la rúbrica» del final del informe: un nivel de 1 a 4 por criterio y **una frase de prueba**. Si hay tiempo, arreglad lo más fácil; lo demás queda como «lo primero que mejoraríamos». Si sobran minutos, empezad el extra.

Qué deberíais ver. `informe_calidad.md` con todos los apartados rellenos y la sección «Revisión con la rúbrica» completa, un `README.md` con los comandos y las versiones, y el cuaderno ejecutándose de arriba abajo sin errores.

Hecho cuando: otra persona podría repetir el cuaderno siguiendo solo el README, y el informe tiene anotados el nivel y la prueba de cada criterio.

Después, 35 minutos de entrega, review y retro (también el día 2): subid los ficheros a `sprint-06/` siguiendo Cómo entregar en GitHub, cada persona con su cuenta. Descargad el cuaderno con *Archivo → Descargar → Descargar .ipynb* (después de *Reiniciar y ejecutar todo*) y subidlo. Los nombres de archivo, **en minúsculas y sin espacios**; el CSV va a `data/` solo si aún no está. Un reparto posible: la persona A sube `informe_calidad.md` («Añado informe de calidad»), la B sube `reto_S06.ipynb` («Añado cuaderno»), la C actualiza `README.md` («Completo el README»). Después, cada persona rellena y sube su autoevaluación y coevaluación. En la *review* (5 minutos por equipo) enseñad: la tabla de defectos (los 3 más graves y por qué), el cuaderno corriendo desde cero y la métrica del *baseline* frente al modelo tonto. Para la retro, pensad: ¿qué defecto nos costó más encontrar? ¿Qué decisión de limpieza discutimos más? ¿Qué haríamos distinto con el siguiente fichero del cliente?

Ayuda 1 · Cómo usar la IA en este reto

La limpieza de datos es un terreno donde la IA ayuda mucho, pero **donde también introduce errores sutiles**. Reglas:

- **Todo lo que dé, lo ejecutáis y lo entendéis.** Si el docente pregunta por una línea, tenéis que saber explicarla.

- **Solo librerías del curso.** No instaléis paquetes extra que os «arreglen» los datos automáticamente.
- **Toda corrección va en el código**, no en el CSV ni en un clic de Excel.
- **Declaradlo** en el README: qué pedisteis y qué revisasteis.
- **Nada de datos personales ni claves** en una IA (los datos de este reto son sintéticos, pero se coge el hábito).

Para qué	Ejemplo de petición	Qué comprobáis
Entender una función de pandas	«Explícame `fillna`, `drop_duplicates` y `to_datetime` con un ejemplo muy pequeño»	Que probáis cada ejemplo y lo entendéis
Depurar un error	«Este código da `KeyError: 'precio'`. El DataFrame tiene estas columnas: *[lista]*. ¿Qué puede pasar?»	Que arregláis la causa, no solo el síntoma
Revisar la fuga	«Aquí está mi pipeline: *[código]*. ¿Hay alguna operación que use el test para aprender?»	Que entendéis **por qué** hay o no fuga
Redactar el informe	«Reescribe esta tabla de defectos de forma más clara»	Que **no cambie cifras ni defectos**

Ayuda 2 · Errores frecuentes (y cómo evitarlos)

Error	Por qué ocurre	Cómo evitarlo
Calcular la mediana con **todos** los datos antes de partir	«Total, es solo la mediana»	Partid primero; imputación y escalado se ajustan **solo con *train***, dentro del `Pipeline`
Borrar filas «raras» sin avisar	Se quieren datos limpios	Cada fila eliminada se **cuenta y se justifica** en el informe
Informe que dice «hay nulos» sin más	Falta el análisis	Para cada defecto: dónde, cuántas filas, causa, tratamiento y razón
Un modelo que «acierta mucho» y se celebra	La exactitud engaña si una clase pesa mucho más	Comparadlo con `DummyClassifier` y mirad F1 o ROC-AUC
El cuaderno solo funciona con las celdas en cierto orden	Se editaron celdas sueltas	*Reiniciar y ejecutar todo* antes de entregar
Convertir las fechas y que salgan nulas o con día y mes cambiados	Hay dos formatos mezclados y se aplica solo uno	Probar cada formato por separado con `errors="coerce"` y comprobar el año mínimo y máximo
Usar columnas que delatan la respuesta o que no se conocen al reservar	Parece que ayudan	Preguntad por cada columna: ¿se conoce **al reservar**?
Que la limpieza cambie el CSV original o la tabla `crudo`	Se modifica la tabla en lugar de una copia	Empezar la función con `.copy()` y no escribir nunca en `crudo`

Ayuda 3 · Si os atascáis

1. **No encontramos más defectos:** probad `df.describe()`, `df.isna().sum()`, `df.duplicated().sum()` y `df[columna].value_counts()` en cada columna, y revisad las ocho categorías de la tabla del paso 2.
2. **No sabemos si algo es un error o un dato real:** apuntadlo como **pregunta para el cliente** y explicad qué hacéis mientras tanto.
3. **Una función de pandas no hace lo que queremos:** probadla con un ejemplo de 3 filas inventadas, y repasad el cuaderno correspondiente y su código explicado. Después, preguntad al equipo y al docente.